

ECC Assignment-1

★ Hadoop Installation Process:

The following are the steps that I used for Hadoop installation:

- Step 1: Set up a Jetstream instance with Ubuntu to host Hadoop.
- Step 2: JDK 11 was updated as a prerequisite for running Hadoop.
- Step 3: Download Hadoop 3.3.6.
- Step 4: Update configuration files (hdfs-site.xml, mapred-site.xml) with important settings for Hadoop's distributed file system and MapReduce framework.
- Step 5: Set environment variables in .bashrc (as shown in Fig 3), including JAVA_HOME and HADOOP_HOME.
- Step 6: Launch Hadoop using the start-all.sh script (shown in Fig 1 & Fig 2). This script initiates NameNode, DataNode, NodeManager, and ResourceManager processes.

```
For more details see sb(1).  
vvp@Ubuntu:~$ su - hadoop  
Password:  
hadoop@Ubuntu:~$ start-all.sh  
WARNING: Attempting to start all Apache Hadoop daemons as hadoop in 10 seconds.  
WARNING: This is not a recommended production deployment configuration.  
WARNING: Use CTRL-C to abort.  
Starting namenodes on [localhost]  
Starting datanodes  
Starting secondary namenodes [Ubuntu]  
Starting resourcemanager  
Starting nodemanagers
```

Fig 1: start-all.h Output

```
Starting nodemanagers  
hadoop@Ubuntu:~$ jps  
3875 NameNode  
4053 DataNode  
4471 ResourceManager  
4587 NodeManager  
4269 SecondaryNameNode  
4959 Jps
```

Fig 2: All processes are running

```

export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/home/hadoop/hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"

```

Fig 3: Environment variables setup in .bashrc

The screenshot shows the Hadoop user interface in a web browser. The browser address bar shows the URL `10.0.2.15:9870/dfshealth.html#tab-overview`. The interface has a green header bar with tabs: **Hadoop**, **Overview**, **Datanodes**, **Datanode Volume Failures**, **Snapshot**, **Startup Progress**, and **Utilities**.

Overview 'localhost:9000' (active)

Started:	Mon Mar 04 01:01:56 -0500 2024
Version:	3.3.6, r1be78238728da9266a4f88195058f08fd012bf9c
Compiled:	Sun Jun 18 04:22:00 -0400 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)
Cluster ID:	CID-24b93488-4f52-4a8c-8953-1c1e60aab6cd
Block Pool ID:	BP-391614948-127.0.1.1-1709439205894

Summary

Security is off.
 Safemode is off.
 127 files and directories, 108 blocks (108 replicated blocks, 0 erasure coded block groups) = 235 total filesystem object(s).
 Heap Memory used 164.16 MB of 372.5 MB Heap Memory. Max Heap Memory is 2.11 GB.
 Non Heap Memory used 51.01 MB of 52.36 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	23.94 GB
Configured Remote Capacity:	0 B
DFS Used:	10.95 MB (0.04%)
Non DFS Used:	15.29 GB
DFS Remaining:	7.41 GB (30.93%)
Block Pool Used:	10.95 MB (0.04%)
DataNodes usages% (Min/Median/Max/stdDev):	0.04% / 0.04% / 0.04% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	0
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Mon Mar 04 01:01:56 -0500 2024

Fig 4: User Interface of Hadoop

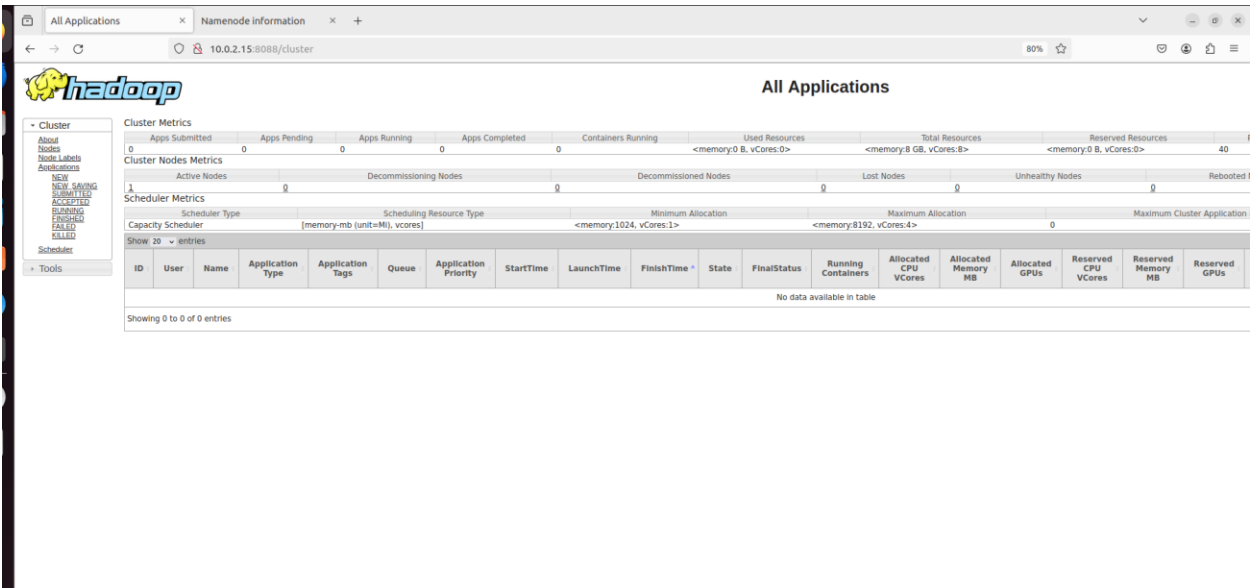


Fig 5: Hadoop Cluster

★ PART 1. Output the top-3 IP addresses with the granularity of an hour

Mapper (mapper_vaishp_part1.py):

- The mapper code (shown in Fig 6.1) searches and extracts IP addresses and hours from an input log file.
- For each match, it prints the hour and IP along with a count, which is then passed to the reducer.

```
import re
import sys

# We are using this regex to extract IP & Hour which is required in this Assignment
IpHourPattern = re.compile(
    '(?P<ip>\d+\.\d+\.\d+\.\d+).*?\d{4}:(?P<hour>\d{2}):\d{2}.*? '
)

InputData = sys.stdin.readline()
while InputData:
    ResultFound = IpHourPattern.search(InputData)
    if ResultFound:
        IPAddress, TimeHour = ResultFound.groups()
        print(f'{TimeHour}, {IPAddress}, 1')
    InputData = sys.stdin.readline()
~
~
```

Fig 6.1 Mapper (mapper_vaishp_part1.py)

Reducer (reducer_vaishp_part1.py):

- The reducer code is shown in Fig 6.2 Reducer (reducer_vaishp_part1.py).
- It aggregates IP address counts for specific hours and sorts them in descending order.
- Finally, the top three IPs are printed, along with their counts for each hour.

```

import sys
from collections import defaultdict

# We use IPCountPerHour, to store IP_Address & their Total Occurrence
IPCountPerHour = defaultdict(lambda: defaultdict(int))

for RecordData in sys.stdin:
    RecordData = RecordData.strip()
    TimeByHour, IPAddress, totaloccurrence = RecordData.split(',')
    CalHour = TimeByHour.split(':')[0]
    IPCountPerHour[CalHour][IPAddress] += int(totaloccurrence)

IPCountPerHour = list(IPCountPerHour.items())
indx = 0

while indx < len(IPCountPerHour):
    CalHour, AllIPs = IPCountPerHour[indx]
    # Sorting the counts in Descending order & Displaying Top 3 IP Addresses
    Top3Ips = sorted(
        AllIPs.items(), key=lambda item: item[1], reverse=True)[:3]
    # Printing Top3 Hour, IPAddresses and TotalCount
    print("-----")

    idx = 0
    while idx < len(Top3Ips):
        IPAddress, total = Top3Ips[idx]
        print(f"Hour: {CalHour}:00 - IP: {IPAddress} - COUNT: {total}")
        idx += 1

    indx += 1

print("-----")
~
~
~

```

6.2 Reducer (reducer_vaishp_part1.py)

Command to run Map Reduce for Part 1:

```

hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_part1.py, reducer_vaishp_part1.py -mapper 'python3 mapper_vaishp_part1.py' -
reducer 'python3 reducer_vaishp_part1.py' -input /input/sample.log -output /output/result1

```

Logs:

- Below (Fig 6.3 Logs of Part 1) are the logs obtained after running the above command.

```

hadoop@Ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-files mapper_valshp_part1.py, reducer_valshp_part1.py \
-mapper 'python3 mapper_valshp_part1.py' \
-reducer 'python3 reducer_valshp_part1.py' \
-input /input/sample.log \
-output /output/result1
packageJobJar: [/tmp/hadoop-unjar7448657446790251475/] [] /tmp/streamjob4124438525952493819.jar tmpDir=null
2024-03-04 01:21:15,625 INFO client.DefaultHARMPFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-04 01:21:15,943 INFO client.DefaultHARMPFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-04 01:21:16,418 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1709532132686_0001
2024-03-04 01:21:18,035 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-04 01:21:18,202 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-04 01:21:18,528 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1709532132686_0001
2024-03-04 01:21:18,528 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-04 01:21:18,971 INFO conf.Configuration: resource-types.xml not found
2024-03-04 01:21:18,971 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-03-04 01:21:19,663 INFO impl.YarnClientImpl: Submitted application application_1709532132686_0001
2024-03-04 01:21:19,745 INFO mapreduce.Job: The url to track the job: http://ubuntu.myguest.virtualbox.org:8088/proxy/application_1709532132686_0001/
2024-03-04 01:21:19,747 INFO mapreduce.Job: Running job: job_1709532132686_0001
2024-03-04 01:21:31,943 INFO mapreduce.Job: Job job_1709532132686_0001 running in uber mode : false
2024-03-04 01:21:31,944 INFO mapreduce.Job: map 0% reduce 0%
2024-03-04 01:21:31,944 INFO mapreduce.Job: map 100% reduce 0%
2024-03-04 01:21:44,697 INFO mapreduce.Job: map 100% reduce 100%
2024-03-04 01:21:44,744 INFO mapreduce.Job: Job job_1709532132686_0001 completed successfully
2024-03-04 01:21:44,894 INFO mapreduce.Job: Counters: 54
File System Counters
  FILE: Number of bytes read=7416
  FILE: Number of bytes written=854252
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=106676
  HDFS: Number of bytes written=250
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0
Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=10882
  Total time spent by all reduces in occupied slots (ms)=2656
  Total time spent by all map tasks (ms)=10882
  Total time spent by all reduce tasks (ms)=2656
  Total vcore-milliseconds taken by all map tasks=10882
  Total vcore-milliseconds taken by all reduce tasks=2656
  Total megabyte-milliseconds taken by all map tasks=11143168
  Total megabyte-milliseconds taken by all reduce tasks=2719744
Map-Reduce Framework
  Map input records=320
  Map output records=319
  Map output bytes=6772
  Map output materialized bytes=7422
  Input split bytes=180
  Combine input records=0
  Combine output records=0
  Reduce input groups=46
  Reduce shuffle bytes=7422
  Reduce input records=319
  Reduce output records=5
  Spilled Records=638
  Shuffled Maps =2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=390
  CPU time spent (ms)=3410
  Physical memory (bytes) snapshot=960765952
  Virtual memory (bytes) snapshot=7720706160
  Total committed heap usage (bytes)=805306368
  Peak Map Physical memory (bytes)=381399040
  Peak Map Virtual memory (bytes)=2573385728
  Peak Reduce Physical memory (bytes)=264650752
  Peak Reduce Virtual memory (bytes)=2576830464
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=106496
File Output Format Counters
  Bytes Written=250
2024-03-04 01:21:44,894 INFO streaming.StreamJob: Output directory: /output/result1

```

Fig 6.3 Logs of Part 1

Command to check output: `hdfs dfs -cat /output/result1/part-00000`

Final Output:

```

hadoop@Ubuntu:~$ hdfs dfs -cat /output/result1/part-00000
-----
Hour: 03:00 - IP: 66.111.54.249 - COUNT: 38
Hour: 03:00 - IP: 5.211.97.39 - COUNT: 36
Hour: 03:00 - IP: 66.249.66.194 - COUNT: 31
-----

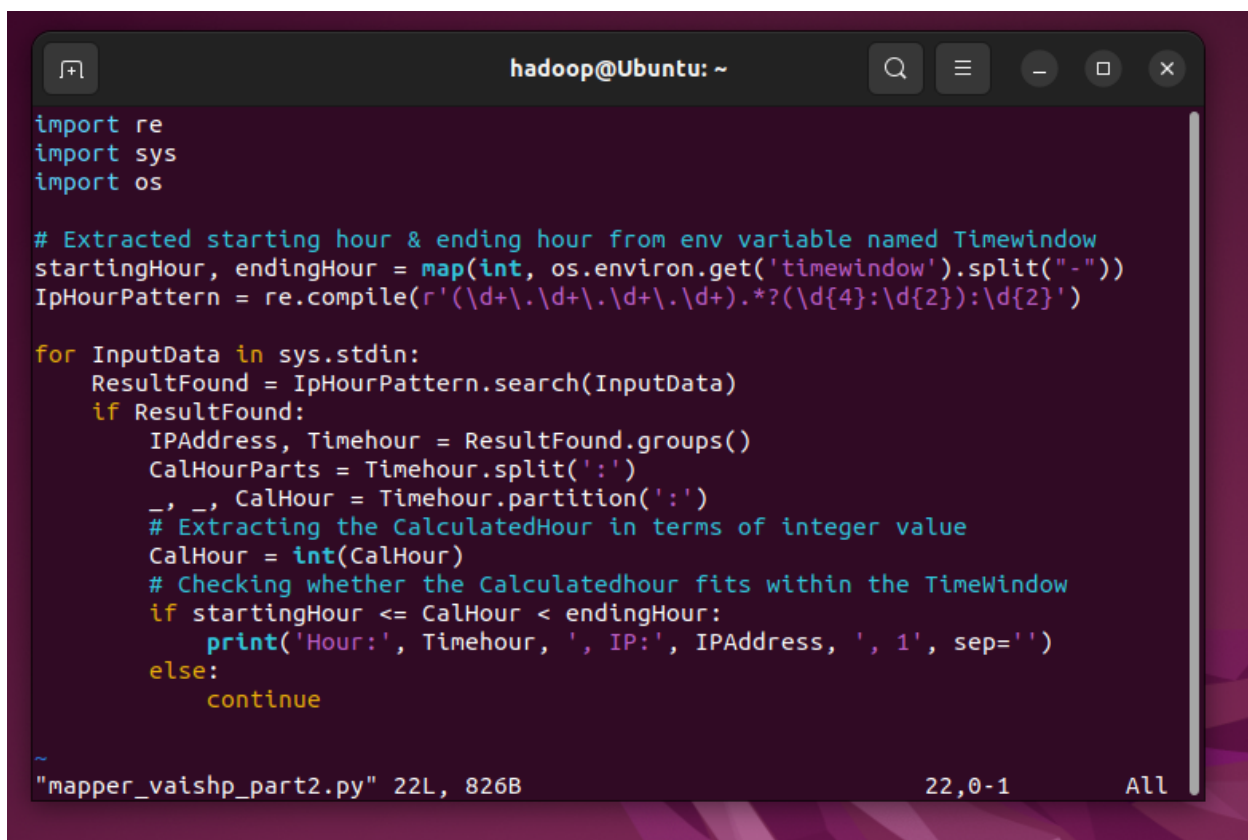
```

Fig: Final Output of Part 1

- ★ **PART 2.1: Make your program like a database search. Your program should be able to accept parameters from users, such as 0-1, which means from time 00:00 to 01:00, and output the Top-3 IP addresses in the given time period.**

Mapper (mapper_vaishp_part2.py):

- The mapper code (Fig 7.1) accepts a command line argument called time window.
- It is used to extract the start and end hours from it.
- Based on this, it filters and prints entries that occur within the time frame specified.



```

hadoop@Ubuntu: ~
import re
import sys
import os

# Extracted starting hour & ending hour from env variable named Timewindow
startingHour, endingHour = map(int, os.environ.get('timewindow').split("-"))
IpHourPattern = re.compile(r'(\d+\.\d+\.\d+\.\d+).*?(\d{4}:\d{2}):\d{2}')

for InputData in sys.stdin:
    ResultFound = IpHourPattern.search(InputData)
    if ResultFound:
        IPAddress, Timehour = ResultFound.groups()
        CalHourParts = Timehour.split(':')
        _, _, CalHour = Timehour.partition(':')
        # Extracting the CalculatedHour in terms of integer value
        CalHour = int(CalHour)
        # Checking whether the Calculatedhour fits within the TimeWindow
        if startingHour <= CalHour < endingHour:
            print('Hour:', Timehour, ', IP:', IPAddress, ', 1', sep='')
        else:
            continue

~
"mapper_vaishp_part2.py" 22L, 826B          22,0-1          All

```

Fig 7.1 Mapper (mapper_vaishp_part2.py)

Reducer (reducer_vaishp_part1.py):

- The reducer code is shown in Fig 7.2 Reducer (reducer_vaishp_part2.py)
- It aggregates IP address counts for specific hours and sorts them in descending order.
- Finally, the top three IPs are printed, along with their counts for each hour.

- I considered two test cases: the first, time window, 01 to 02, has no IP addresses, and the second, 01 to 04, has IP addresses.

```
import sys
from collections import defaultdict

# Used IPCountPerHour, to store IPAddress & their Total Occurrences
IPCountPerHour = defaultdict(lambda: defaultdict(int))

for RecordData in sys.stdin:
    RecordData = RecordData.strip()
    TimeByHour, IPAddress, totaloccurrence = RecordData.split(',')
    # Updating the IPAddress with count of occurrences for the specific hour
    IPCountPerHour[TimeByHour][IPAddress] += int(totaloccurrence)

for TimeByHour, AllIPs in IPCountPerHour.items():
    CalHourParts = TimeByHour.split(':')
    CalHour = CalHourParts[2]
    # Sorting the counts in Descending order & Displaying Top 3 IP Addresses
    Top3Ips = sorted(AllIPs.items(), key=lambda x: x[1], reverse=True)[:3]

    # Printing Top3 Hour, IPAddresses and TotalCount
    idx = 0
    while idx < len(Top3Ips):
        IPAddress, total = Top3Ips[idx]
        print(f"Hour: {CalHour}:00 - IP: {IPAddress} - COUNT: {total}")
        idx += 1

print("-----")
~
~
```

Fig 7.2 Reducer (reducer_vaishp_part2.py)

Command to run Map Reduce for Part 2 – Test Case 1 Time Window 01 to 02:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_part2.py, reducer_vaishp_part2.py -mapper 'python3 mapper_vaishp_part2.py' -
reducer 'python3 reducer_vaishp_part2.py' -input /input/sample.log -output /output/result2 -cmdenv
timewindow="01-02"
```

Logs:

- Below (Fig 7.3 Logs of Part 2-Test Case 1) are the logs obtained after running the above command.


```

hadoop@Ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files mapper_vaishp_part2.py, reducer_vaishp_part2.py -mapper 'python3
mapper_vaishp_part2.py' -reducer 'python3 reducer_vaishp_part2.py' -input /input/sample.log -output /output/result2 -cdenv ttimeWindow="01-02"
packageJobJar: [/tmp/hadoop-unjar6123658060880878805/] [] /tmp/streamjob3289925674019914520.jar tmpDir=null
2024-03-04 02:01:52,918 INFO client.DefaultHARMFallOverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-04 02:01:53,224 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1709532132686_0002
2024-03-04 02:01:53,595 INFO mapreduce.JobResourceUploader: Total input files to process : 1
2024-03-04 02:01:54,614 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-04 02:01:54,967 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-04 02:01:55,466 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1709532132686_0002
2024-03-04 02:01:55,468 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-04 02:01:55,954 INFO conf.Configuration: resource-types.xml not found
2024-03-04 02:01:55,955 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-03-04 02:01:56,175 INFO impl.YarnClientImpl: Submitted application application_1709532132686_0002
2024-03-04 02:01:56,333 INFO mapreduce.Job: The url to track the job: http://ubuntu.myquest.virtualbox.org:8088/proxy/application_1709532132686_0002/
2024-03-04 02:01:56,336 INFO mapreduce.Job: Running job: job_1709532132686_0002
2024-03-04 02:02:08,068 INFO mapreduce.Job: Job job_1709532132686_0002 running in uber mode : false
2024-03-04 02:02:08,070 INFO mapreduce.Job: map 0% reduce 0%
2024-03-04 02:02:16,727 INFO mapreduce.Job: map 100% reduce 0%
2024-03-04 02:02:22,812 INFO mapreduce.Job: map 100% reduce 100%
2024-03-04 02:02:23,867 INFO mapreduce.Job: Job job_1709532132686_0002 completed successfully
2024-03-04 02:02:24,043 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=6
    FILE: Number of bytes written=839483
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=106676
    HDFS: Number of bytes written=0
    HDFS: Number of read operations=11
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=2
    Launched reduce tasks=1
    Data-local map tasks=2
    Total time spent by all maps in occupied slots (ms)=13364
    Total time spent by all reduces in occupied slots (ms)=3363
    Total time spent by all map tasks (ms)=13364
    Total time spent by all reduce tasks (ms)=3363
    Total vcore-milliseconds taken by all map tasks=13364
    Total vcore-milliseconds taken by all reduce tasks=3363
    Total megabyte-milliseconds taken by all map tasks=13684736
    Total megabyte-milliseconds taken by all reduce tasks=3443712
  Map-Reduce Framework
    Map input records=320
    Map output records=0
    Map output bytes=0
    Map output materialized bytes=12
    Input split bytes=180
    Combine input records=0
    Combine output records=0
    Reduce input groups=0
    Reduce shuffle bytes=12
    Reduce input records=0
    Reduce output records=0
    Spilled Records=0
    Shuffled Maps =2
    Failed Shuffles=0
    Merged Map outputs=2
    GC time elapsed (ms)=680
    CPU time spent (ms)=4400
    Physical memory (bytes) snapshot=1021145088
    Virtual memory (bytes) snapshot=7712284672
    Total committed heap usage (bytes)=876085248
    Peak Map Physical memory (bytes)=380567552
    Peak Map Virtual memory (bytes)=2570199040
    Peak Reduce Physical memory (bytes)=261931008
    Peak Reduce Virtual memory (bytes)=2572492800
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=106496
  File Output Format Counters
    Bytes Written=0
2024-03-04 02:02:24,044 INFO streaming.StreamJob: Output directory: /output/result2

```

Fig 7.3 Logs of Part 2-Test Case 1

Command to check output: `hdfs dfs -cat /output/result2/part-00000`

Final Output Part 2- Test Case 1:

```

hadoop@Ubuntu:~$ hdfs dfs -cat /output/result2/part-00000
hadoop@Ubuntu:~$

```

Fig 7.4 Final Output part 2- Test Case 1

Command to run Map Reduce for Part 2 – Test Case 2 Time Window 01 to 04:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_part2.py, reducer_vaishp_part2.py -mapper 'python3 mapper_vaishp_part2.py' -
reducer 'python3 reducer_vaishp_part2.py' -input /input/sample.log -output /output/result2ds -cmdenv
timewindow="01-04"
```

Logs:

- Below (Fig 7.5 Logs of Part 2-Test Case 2) are the logs obtained after running the above command.

```
hadoop@Ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files mapper_vaishp_part2.py, reducer_vaishp_part2.py -mapper 'python3
mapper_vaishp_part2.py' -reducer 'python3 reducer_vaishp_part2.py' -input /input/sample.log -output /output/result2ds -cmdenv timewindow="01-04"
packageJobJar: [/tmp/hadoop-unjar1753509340641368556/] [] /tmp/streamjob7820339928538942576.jar tmpDir=null
2024-03-04 02:44:14,676 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-04 02:44:14,849 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-04 02:44:15,137 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1709532132686_0013
2024-03-04 02:44:15,525 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-04 02:44:15,640 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-04 02:44:15,805 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1709532132686_0013
2024-03-04 02:44:15,805 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-04 02:44:16,047 INFO conf.Configuration: resource-types.xml not found
2024-03-04 02:44:16,047 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-03-04 02:44:16,132 INFO impl.YarnClientImpl: Submitted application application_1709532132686_0013
2024-03-04 02:44:16,239 INFO mapreduce.Job: The url to track the job: http://Ubuntu.myquest.virtualbox.org:8088/proxy/application_1709532132686_0013/
2024-03-04 02:44:16,241 INFO mapreduce.Job: Running job: job_1709532132686_0013
2024-03-04 02:44:22,433 INFO mapreduce.Job: Job job_1709532132686_0013 running in uber mode : false
2024-03-04 02:44:22,435 INFO mapreduce.Job: map 0% reduce 0%
2024-03-04 02:44:27,555 INFO mapreduce.Job: map 100% reduce 0%
2024-03-04 02:44:32,629 INFO mapreduce.Job: map 100% reduce 100%
2024-03-04 02:44:33,652 INFO mapreduce.Job: Job job_1709532132686_0013 completed successfully
2024-03-04 02:44:33,798 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=11563
    FILE: Number of bytes written=862600
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=106676
    HDFS: Number of bytes written=373
    HDFS: Number of read operations=11
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=2
    Launched reduce tasks=1
    Data-local map tasks=2
    Total time spent by all maps in occupied slots (ms)=6383
    Total time spent by all reduces in occupied slots (ms)=2415
    Total time spent by all map tasks (ms)=6383
    Total time spent by all reduce tasks (ms)=2415
    Total vcore-milliseonds taken by all map tasks=6383
    Total vcore-milliseonds taken by all reduce tasks=2415
    Total megabyte-milliseonds taken by all map tasks=6536192
    Total megabyte-milliseonds taken by all reduce tasks=2472960
  Map-Reduce Framework
    Map input records=320
    Map output records=319
```

Fig 7.5 Logs of Part 2-Test Case 2

Command to check output: `hdfs dfs -cat /output/result2ds/part-00000`

Final Output Part 2- Test Case 2:

```
hadoop@Ubuntu:~$ hdfs dfs -cat /output/result2ds/part-00000
Hour: 03:00 - IP: IP:66.111.54.249 - COUNT: 38
Hour: 03:00 - IP: IP:5.211.97.39 - COUNT: 36
Hour: 03:00 - IP: IP:66.249.66.194 - COUNT: 31
-----
```

Fig 7.6 Final Output part 2- Test Case 1

- ★ PART 2.2: Run it along with three other examples, WordCount, Sort, Grep, at the same time, and test fair and capacity schedulers.

Run all the task:

- To run all tasks in parallel, I created a script called runAllcode.sh, which includes wordcount, sort, database search, and grep.
- I ran these jobs individually and added them to a script with & at the end to run in the background concurrently.
- Mapper and reducer code and output for each of the above tasks.
- I added each queue name to each command with the flag `-D mapreduce.job.queueName`. Below is a screenshot of runAll.sh.

```
hadoop@Ubuntu: ~
#!/bin/bash

# Database Search Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=DatabaseSearch -D mapred.job.priority=VERY_HIGH \
-files mapper_vaishp_part2.py,reducer_vaishp_part2.py \
-mapper 'python3 mapper_vaishp_part2.py' \
-reducer 'python3 reducer_vaishp_part2.py' \
-input /input/sample.log \
-output /output/result2ds \
-cmdenv timewindow="01-02" &

# Word Count Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=WordCount -D mapred.job.priority=HIGH \
-files mapper_vaishp_wc.py,reducer_vaishp_wc.py \
-mapper 'python3 mapper_vaishp_wc.py' \
-reducer 'python3 reducer_vaishp_wc.py' \
-input /input/sample.log \
-output /output/result2wc &

# Sort Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=Sort -D mapred.job.priority=NORMAL \
-files mapper_vaishp_sort.py,reducer_vaishp_sort.py \
-mapper 'python3 mapper_vaishp_sort.py' \
-reducer 'python3 reducer_vaishp_sort.py' \
-input /input/sample.log \
-output /output/result2sort &

# Grep Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=Grep -D mapred.job.priority=LOW \
-files mapper_vaishp_grep.py,reducer_vaishp_grep.py \
-mapper 'python3 mapper_vaishp_grep.py' \
-reducer 'python3 reducer_vaishp_grep.py' \
-input /input/sample.log \
-output /output/result2grep \
-cmdenv search_pattern="sin" &

wait

echo "-----ALL TASK COMPLETED-----"
```

Fig : runAll.sh File

★ Sort Task:

Mapper Sort (mapper_vaishp_sort.py)

- Extracts the first word as IP, also the remaining words as string.
- Prints the IP Addresses and remaining words in separate formats and sends it to the reducer.

```
import sys

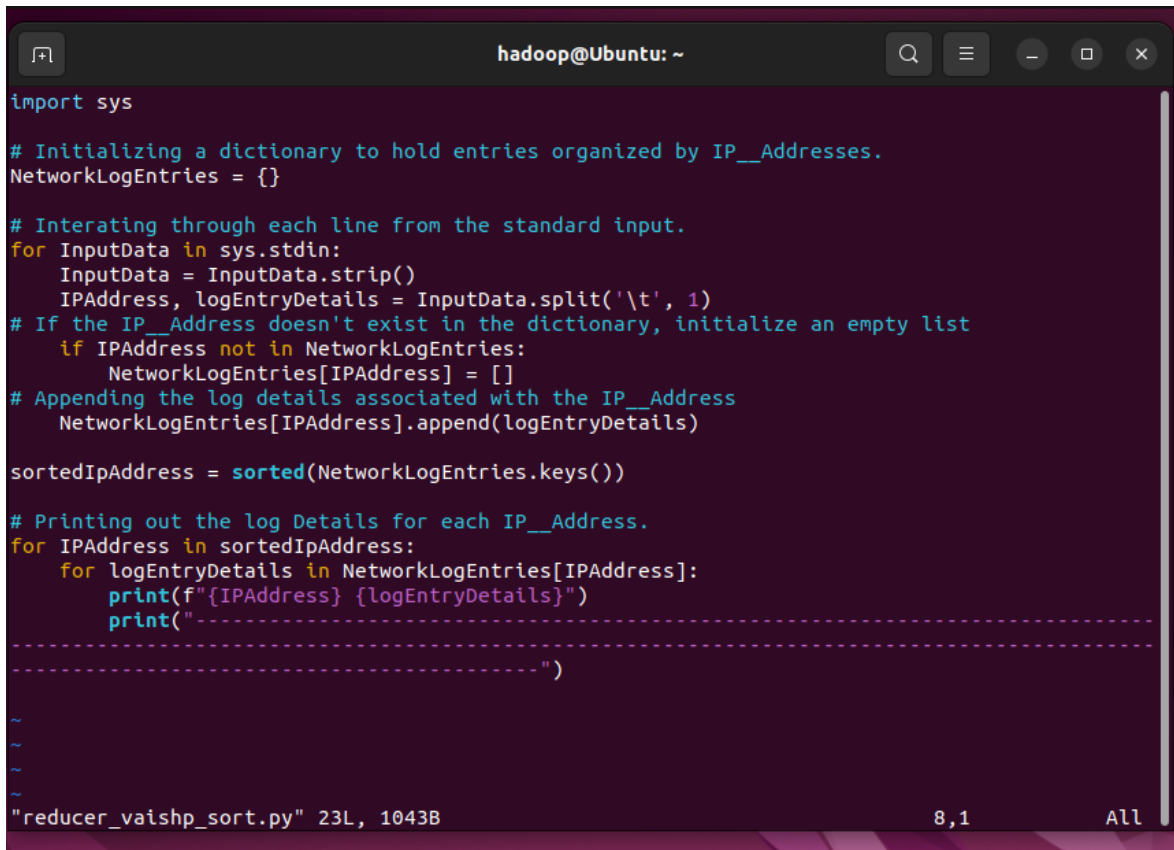
# Iterating through each line from the standard input.
for InputData in sys.stdin:
    # Eliminate the whitespace at the front and rear.
    CleanedLine = InputData.strip()
    # Based on the spaces, divide the line into words.
    Components = CleanedLine.split()

    # Verify that the line is not empty.
    if Components:
        IPAddress, *remainingParts = Components
        # Print the IP address and the remaining line after that.
        print(f"{IPAddress}\t{' '.join(remainingParts)}")
```

Fig 8.1 Mapper Sort (mapper_vaishp_sort)

Reducer Sort (reducer_vaishp_sort.py)

- It stores log entries in a dictionary, sorts them, and prints the IP address alongside the corresponding log entries.



```
hadoop@Ubuntu: ~  
import sys  
  
# Initializing a dictionary to hold entries organized by IP__Addresses.  
NetworkLogEntries = {}  
  
# Iterating through each line from the standard input.  
for InputData in sys.stdin:  
    InputData = InputData.strip()  
    IPAddress, logEntryDetails = InputData.split('\t', 1)  
    # If the IP__Address doesn't exist in the dictionary, initialize an empty list  
    if IPAddress not in NetworkLogEntries:  
        NetworkLogEntries[IPAddress] = []  
    # Appending the log details associated with the IP__Address  
    NetworkLogEntries[IPAddress].append(logEntryDetails)  
  
sortedIpAddress = sorted(NetworkLogEntries.keys())  
  
# Printing out the log Details for each IP__Address.  
for IPAddress in sortedIpAddress:  
    for logEntryDetails in NetworkLogEntries[IPAddress]:  
        print(f"{IPAddress} {logEntryDetails}")  
        print("-----")  
    print("-----")  
  
~  
~  
~  
~  
"reducer_vaishp_sort.py" 23L, 1043B 8,1 All
```

Fig 8.2 Reducer Sort (reducer_vaishp_sort)

Command to run Map Reduce:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files  
mapper_vaishp_sort.py, reducer_vaishp_sort.py -mapper 'python3 mapper_vaishp_sort.py' -reducer  
'python3 reducer_vaishp_sort.py' -input /input/sample.log -output /output/result2sort
```

Logs:

- Below (Fig 8.3 Logs of Sort) are the logs obtained after running the above command.

```

hadoop@Ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files mapper_vaishp_sort.py, reducer_vaishp_sort.py -mapper 'python3 mapper_vaishp_sort.py' -reducer 'python3 reducer_vaishp_sort.py' -input /input/sample.log -output /output/result2sort
packageJobJar: [/tmp/hadoop-unjar579900557831485204/] [] /tmp/streamjob3059507122664772316.jar tmpDir=null
2024-03-09 17:00:47,759 INFO client.DefaultHadoopFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 17:00:47,923 INFO client.DefaultHadoopFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 17:00:48,176 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1710020902181_0002
2024-03-09 17:00:48,625 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-09 17:00:48,755 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-09 17:00:48,935 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1710020902181_0002
2024-03-09 17:00:48,935 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-09 17:00:49,111 INFO conf.Configuration: resource-types.xml not found
2024-03-09 17:00:49,111 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-03-09 17:00:49,204 INFO impl.YarnClientImpl: Submitted application application_1710020902181_0002
2024-03-09 17:00:49,265 INFO mapreduce.Job: The url to track the job: http://Ubuntu.myguest.virtualbox.org:8088/proxy/application_1710020902181_0002/
2024-03-09 17:00:49,272 INFO mapreduce.Job: Running job: job_1710020902181_0002
2024-03-09 17:00:55,426 INFO mapreduce.Job: Job job_1710020902181_0002 running in uber mode : false
2024-03-09 17:00:55,427 INFO mapreduce.Job: map 0% reduce 0%
2024-03-09 17:01:00,518 INFO mapreduce.Job: map 100% reduce 0%
2024-03-09 17:01:06,621 INFO mapreduce.Job: map 100% reduce 100%
2024-03-09 17:01:07,688 INFO mapreduce.Job: Job job_1710020902181_0002 completed successfully
2024-03-09 17:01:07,759 INFO mapreduce.Job: Counters: 54
File System Counters
  FILE: Number of bytes read=104132
  FILE: Number of bytes written=1047672
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=106676
  HDFS: Number of bytes written=171622
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0
Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=5934
  Total time spent by all reduces in occupied slots (ms)=2960
  Total time spent by all map tasks (ms)=5934
  Total time spent by all reduce tasks (ms)=2960
  Total vcore-millisecs taken by all map tasks=5934
  Total vcore-millisecs taken by all reduce tasks=2960
  Total megabyte-millisecs taken by all map tasks=6076416

```

Fig 8.3 Logs of Sort

Command to check output: `hdfs dfs -cat /output/result2sort/part-00000`

Final Output:

```

hadoop@Ubuntu:~$ hdfs dfs -cat /output/result2sort/part-00000
104.194.24.33 - - [22/Jun/2019:03:57:07 +0330] "GET /amp-helper-frame.html?appId=a624a1c1-0c93-466a-a546-e146710f97e6&parentOrigin=https://www-zanbil-lr.cdn.ampproject.org HTTP/1.1" 200 133 "https://www-zanbil-lr.cdn.ampproject.org/v/s/www.zanbil.ir/n/article/594/-?d9%87%09%86%D8%B1-%D9%85%D8%AC%D8%B3%D9%85%D9%87-%D8%B3%D8%7%D8%B2%D8%8C?amp_js_v=a2&amp_gsa=1&usqp=mq33IAQCCEAE3D" "Mozilla/5.0 (Linux; Android 8.0.0; SM-G955F) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.99 Mobile Safari/537.36" "-"
-----
157.55.39.245 - - [22/Jun/2019:03:56:57 +0330] "GET /product/32428/62788/d9%84%D9%BE-%D8%AA%D8%A7%D9%BE-%D8%A7%D8%8C%D8%B3%D9%88%D8%B3-%D9%85%D8%AF%D9%84-Asus-X541UV HTTP/1.1" 200 44052 "-" "Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)" "-"
-----
157.55.39.245 - - [22/Jun/2019:03:56:54 +0330] "GET /blog/sports/gyn-and-fitness/xdbxb7-%d8a7%d8b4%d8%aa%d8%a8%d8a7%d9%87-%d8xb1%d8a7%db%8c%d8%ac-%d8%af%d8%b1-%d8%aa%d9%85%d8%b1%d8%8c%d9%86%d8a7%d8%aa-%d8%a8%d8%af%d9%86%d8%b3%d8a7%d8%b2%db%8c/ HTTP/1.1" 200 25639 "-" "Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)" "-"
-----
157.55.39.245 - - [22/Jun/2019:03:56:51 +0330] "GET /filter/b1,b103,b105,b111,b122,b130,b212,b552,b60,b718,b98 HTTP/1.1" 200 38051 "-" "Mozilla/5.0 (compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm)" "-"
-----
17.58.102.43 - - [22/Jun/2019:03:56:55 +0330] "GET /filter?f=b211%2Cb502%2Cb229%2Cb99%2Cb507%2Cb122%2Cb109%2Cb46%2Cb176%2Cb219%2Cb577%2Cb261%2Cb18&page=3&o=b HTTP/1.1" 302 0 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_1) AppleWebKit/600.2.5 (KHTML, like Gecko) Version/8.0.2 Safari/600.2.5 (Applebot/0.1; +http://www.apple.com/go/applebot)" "-"
-----
17.58.102.43 - - [22/Jun/2019:03:56:57 +0330] "GET /filter/b211%2Cb502%2Cb229%2Cb99%2Cb507%2Cb122%2Cb109%2Cb46%2Cb176%2Cb219%2Cb577%2Cb261%2Cb17?page=3 HTTP/1.1" 200 38542 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_1) AppleWebKit/600.2.5 (KHTML, like Gecko) Version/8.0.2 Safari/600.2.5 (Applebot/0.1; +http://www.apple.com/go/applebot)" "-"
-----
17.58.102.43 - - [22/Jun/2019:03:56:35 +0330] "GET /filter/b571%2Cb288%2Cb70%2Cb57%2Cb6%2Cb355%2Cb485%2Cb87%2Cb551%2Cb148 HTTP/1.1" 200 37759 "-" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_10_1) AppleWebKit/600.2.5 (KHTML, like Gecko) Version/8.0.2 Safari/600.2.5 (Applebot/0.1; +http://www.apple.com/go/applebot)" "-"
-----
172.20.2.174 - - [22/Jun/2019:03:56:49 +0330] "GET /site/ping HTTP/1.1" 200 4 "-" "-" "-"
-----
173.249.54.67 - - [22/Jun/2019:03:56:30 +0330] "GET /image/32745?name=ms506-14.jpg&wh=200x200 HTTP/1.1" 200 3901 "-" "Dalvik/2.1.0 (Linux; U; Android 8.0.0; PRA-LA1 Build/HONORPRA-LA1)" "-"
-----

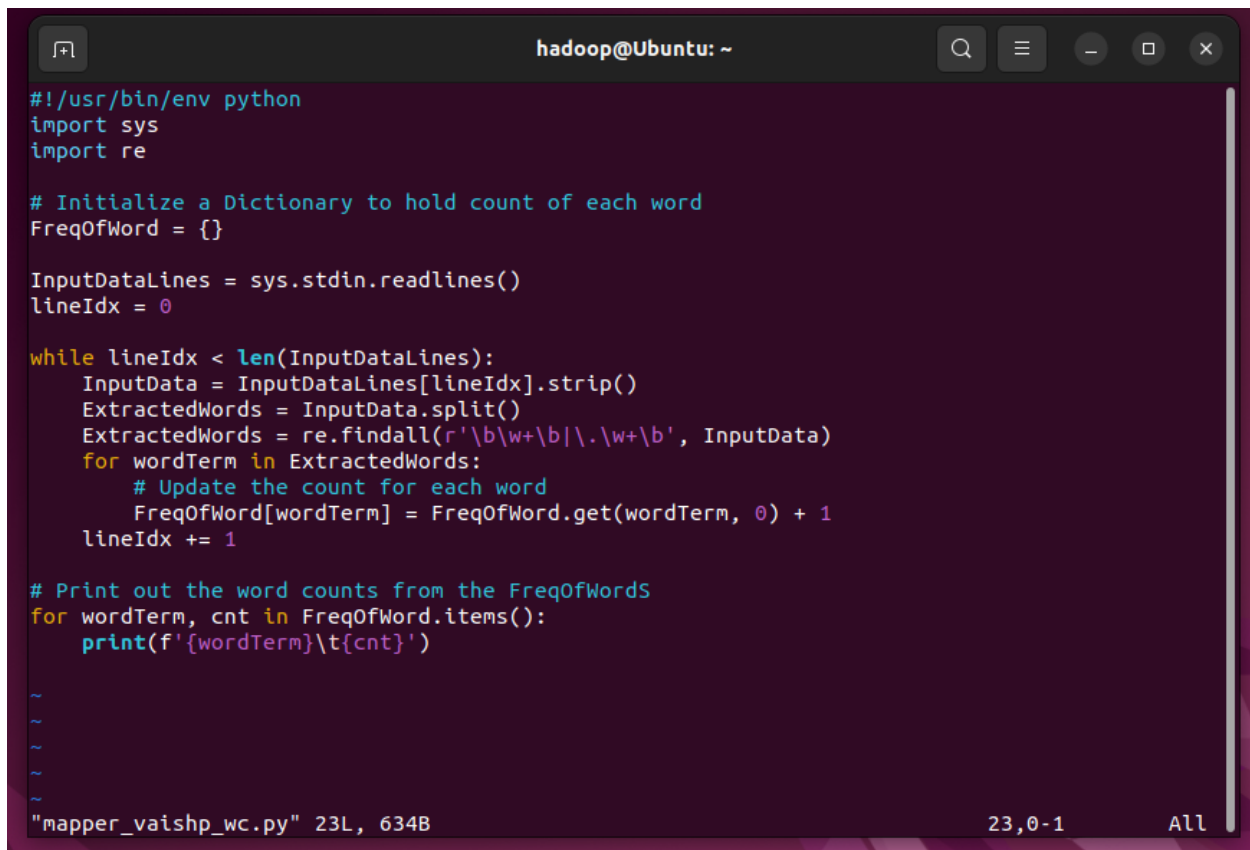
```

Fig 8.4 Final Output for Sort

★ Word Count Task:

Mapper (mapper_vaishp_wc.py):

- It uses a dictionary to store word counts and sends them to the reducer.



```
#!/usr/bin/env python
import sys
import re

# Initialize a Dictionary to hold count of each word
FreqOfWord = {}

InputDataLines = sys.stdin.readlines()
lineIdx = 0

while lineIdx < len(InputDataLines):
    InputData = InputDataLines[lineIdx].strip()
    ExtractedWords = InputData.split()
    ExtractedWords = re.findall(r'\b\w+\b|\.\w+\b', InputData)
    for wordTerm in ExtractedWords:
        # Update the count for each word
        FreqOfWord[wordTerm] = FreqOfWord.get(wordTerm, 0) + 1
    lineIdx += 1

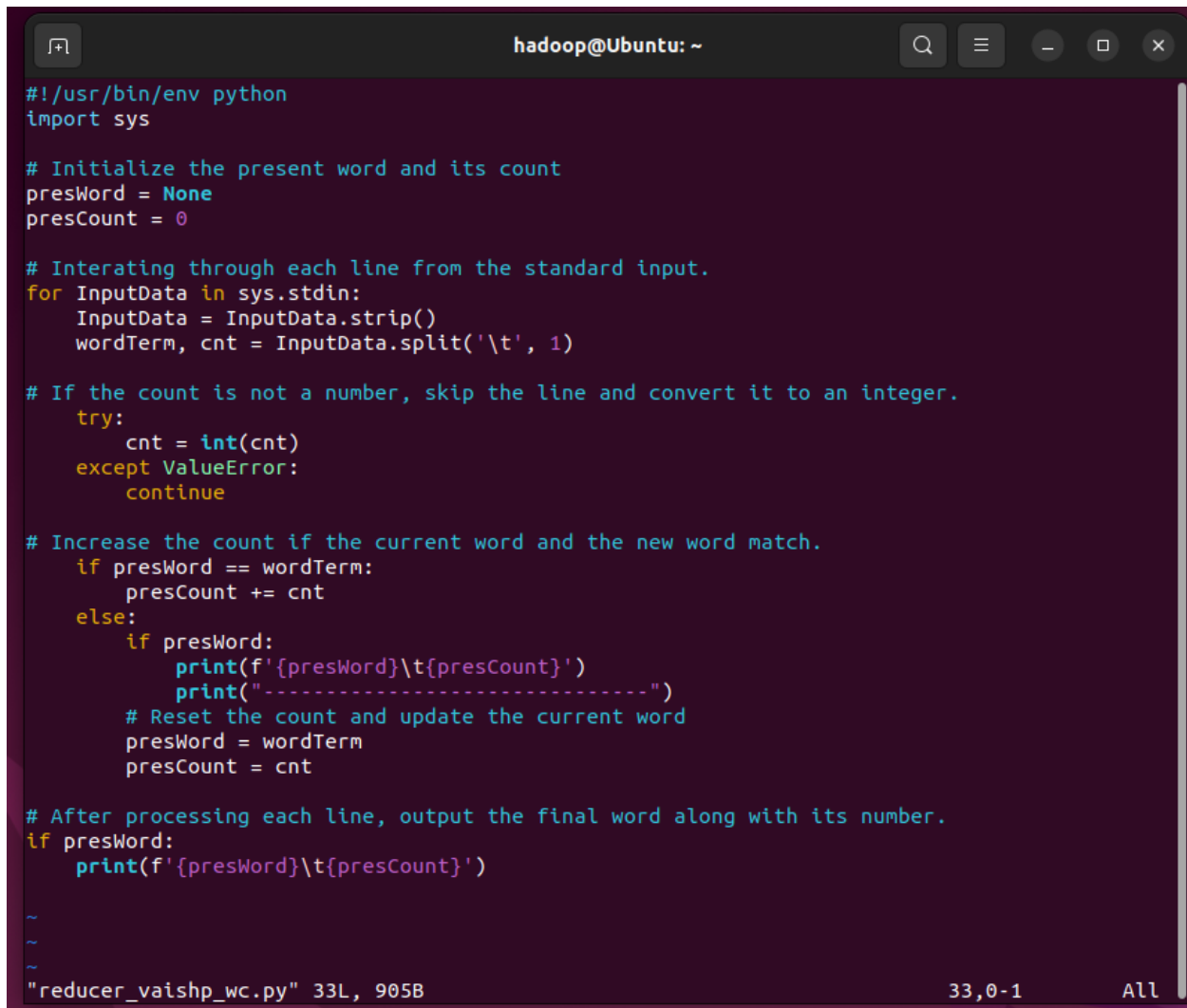
# Print out the word counts from the FreqOfWords
for wordTerm, cnt in FreqOfWord.items():
    print(f'{wordTerm}\t{cnt}')

~
~
~
~
~
"mapper_vaishp_wc.py" 23L, 634B 23,0-1 All
```

Fig 9.1 Mapper for Word Count

Reducer (reducer_vaishp_wc.py)

- Consolidates word counts by processing pairs of words.
- It adds the count of each word and prints it when the word changes.



```
hadoop@Ubuntu: ~
#!/usr/bin/env python
import sys

# Initialize the present word and its count
presWord = None
presCount = 0

# Iterating through each line from the standard input.
for InputData in sys.stdin:
    InputData = InputData.strip()
    wordTerm, cnt = InputData.split('\t', 1)

    # If the count is not a number, skip the line and convert it to an integer.
    try:
        cnt = int(cnt)
    except ValueError:
        continue

    # Increase the count if the current word and the new word match.
    if presWord == wordTerm:
        presCount += cnt
    else:
        if presWord:
            print(f'{presWord}\t{presCount}')
            print("-----")
            # Reset the count and update the current word
            presWord = wordTerm
            presCount = cnt

# After processing each line, output the final word along with its number.
if presWord:
    print(f'{presWord}\t{presCount}')

~
~
~
"reducer_vaishp_wc.py" 33L, 905B 33,0-1 All
```

Fig 9.2 Reducer for Word Count

Command to run Map Reduce:

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_wc.py, reducer_vaishp_wc.py -mapper 'python3 mapper_vaishp_wc.py' -reducer
'python3 reducer_vaishp_wc.py' -input /input/sample.log -output /output/result2wc
```

Logs:

- Below (Fig 9.3 Logs of Sort) are the logs obtained after running the above command.

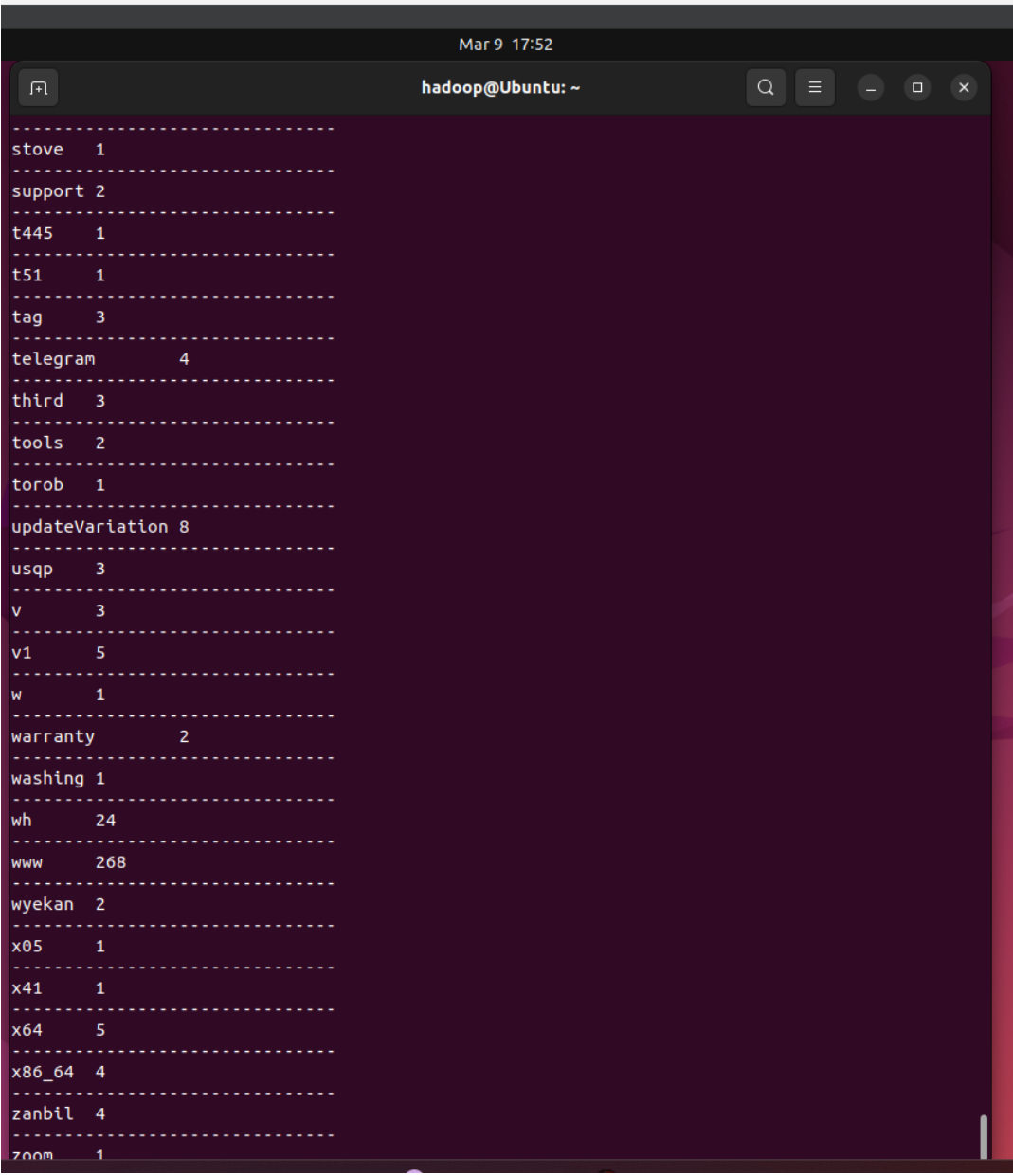

```

hadoop@ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files mapper_vaishp_wc.py,reducer_vaishp_wc.py -mapper 'python3 mapper_vaishp_wc.py' -reducer 'python3 reducer_vaishp_wc.py' -input /input/sample.log -output /output/result2wc
packageJobJar: [/tmp/hadoop-unjar1072739790727558767/] [] /tmp/streamjob2689792407509637590.jar tmpDir=null
2024-03-09 17:42:46,712 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 17:42:46,891 INFO client.DefaultNoHARMFalloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 17:42:47,140 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_1710020902181_0003
2024-03-09 17:42:47,528 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-09 17:42:48,454 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-09 17:42:49,048 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1710020902181_0003
2024-03-09 17:42:49,048 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-09 17:42:49,233 INFO conf.Configuration: resource-types.xml not found
2024-03-09 17:42:49,234 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2024-03-09 17:42:49,347 INFO impl.YarnClientImpl: Submitted application application_1710020902181_0003
2024-03-09 17:42:49,386 INFO mapreduce.Job: The url to track the job: http://Ubuntu.mygquest.virtualbox.org:8088/proxy/application_1710020902181_0003/
2024-03-09 17:42:49,388 INFO mapreduce.Job: Running job: job_1710020902181_0003
2024-03-09 17:42:56,640 INFO mapreduce.Job: Job job_1710020902181_0003 running in uber mode : false
2024-03-09 17:42:56,641 INFO mapreduce.Job: map 0% reduce 0%
2024-03-09 17:43:03,931 INFO mapreduce.Job: map 100% reduce 0%
2024-03-09 17:43:09,989 INFO mapreduce.Job: map 100% reduce 100%
2024-03-09 17:43:11,009 INFO mapreduce.Job: Job job_1710020902181_0003 completed successfully
2024-03-09 17:43:11,114 INFO mapreduce.Job: Counters: 54
  File System Counters
    FILE: Number of bytes read=18280
    FILE: Number of bytes written=875911
    FILE: Number of read operations=0
    FILE: Number of large read operations=0
    FILE: Number of write operations=0
    HDFS: Number of bytes read=186676
    HDFS: Number of bytes written=55812
    HDFS: Number of read operations=11
    HDFS: Number of large read operations=0
    HDFS: Number of write operations=2
    HDFS: Number of bytes read erasure-coded=0
  Job Counters
    Launched map tasks=2
    Launched reduce tasks=1
    Data-local map tasks=2
    Total time spent by all maps in occupied slots (ms)=10328
    Total time spent by all reduces in occupied slots (ms)=2836
    Total time spent by all map tasks (ms)=10328
    Total time spent by all reduce tasks (ms)=2836
    Total vcore-milliseonds taken by all map tasks=10328
    Total vcore-milliseonds taken by all reduce tasks=2836
    Total megabyte-milliseonds taken by all map tasks=10575872
    Total megabyte-milliseonds taken by all reduce tasks=2904064
  Map-Reduce Framework
    Map input records=320
    Map output records=1876
    Map output bytes=14522
    Map output materialized bytes=18286
    Input split bytes=180
    Combine input records=0
    Combine output records=0
    Reduce input groups=1370
    Reduce shuffle bytes=18286
    Reduce input records=1876
    Reduce output records=2739
    Spilled Records=3752
    Shuffled Maps =2
    Failed Shuffles=0
    Merged Map outputs=2
    GC time elapsed (ms)=560
    CPU time spent (ms)=3730
    Physical memory (bytes) snapshot=973000704
    Virtual memory (bytes) snapshot=7718744064
    Total committed heap usage (bytes)=853540864
    Peak Map Physical memory (bytes)=381095936
    Peak Map Virtual memory (bytes)=2571431936
    Peak Reduce Physical memory (bytes)=211865600
    Peak Reduce Virtual memory (bytes)=2576207872
  Shuffle Errors
    BAD_ID=0
    CONNECTION=0
    IO_ERROR=0
    WRONG_LENGTH=0
    WRONG_MAP=0
    WRONG_REDUCE=0
  File Input Format Counters
    Bytes Read=106496
  File Output Format Counters
    Bytes Written=55812
2024-03-09 17:43:11,115 INFO streaming.StreamJob: Output directory: /output/result2wc

```

Fig 9.3 Logs of Word Count

Command to check output: hdfs dfs -cat /output/result2wc/part-00000

Final Output:A terminal window titled 'hadoop@Ubuntu: ~' with a search icon, menu icon, and window control buttons. The terminal displays a word count output with words and their frequencies, separated by dashed lines. The words are listed in ascending order of frequency.

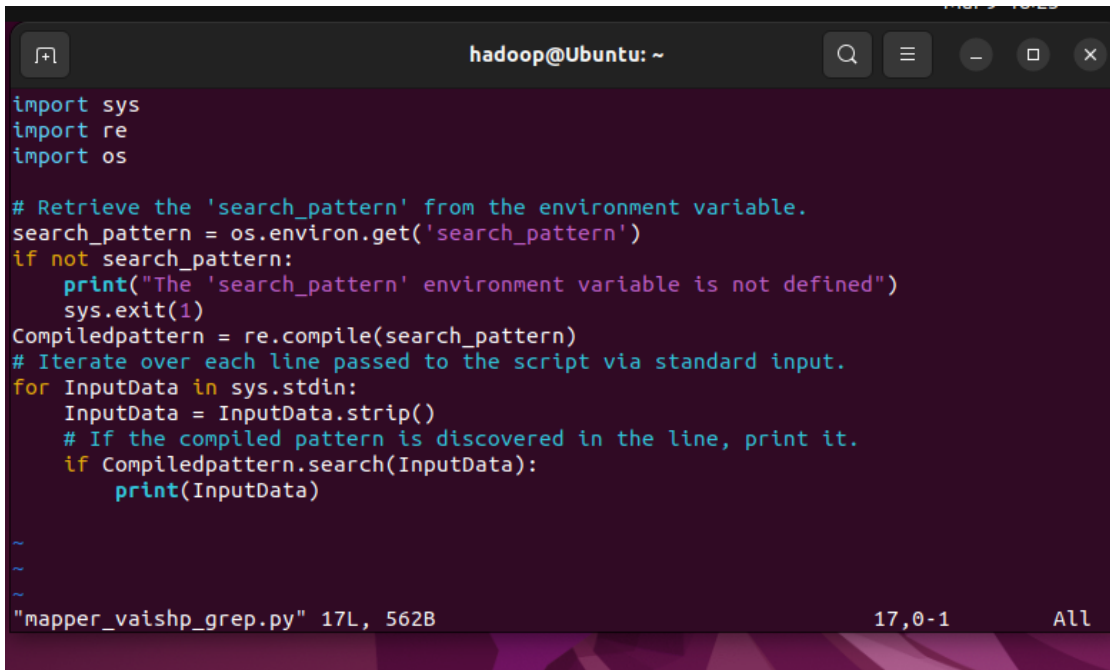
```
-----
stove 1
-----
support 2
-----
t445 1
-----
t51 1
-----
tag 3
-----
telegram 4
-----
third 3
-----
tools 2
-----
torob 1
-----
updateVariation 8
-----
usqp 3
-----
v 3
-----
v1 5
-----
w 1
-----
warranty 2
-----
washing 1
-----
wh 24
-----
www 268
-----
wyekan 2
-----
x05 1
-----
x41 1
-----
x64 5
-----
x86_64 4
-----
zanbil 4
-----
zoom 1
```

Fig 9.4 Final Output Word Count

Grep Task:

Mapper (mapper_vaishp_grep.py):

- It checks if a search pattern entered by the user via command line matches.
- If the line matches the search pattern, it is passed to the reducer.



```
import sys
import re
import os

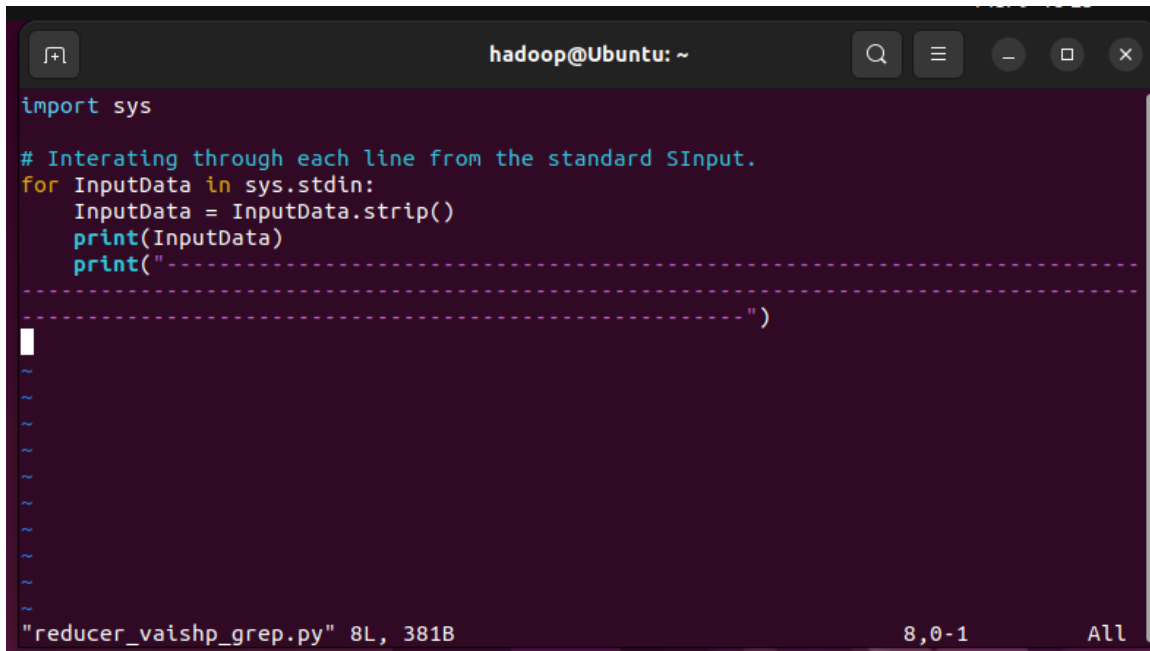
# Retrieve the 'search_pattern' from the environment variable.
search_pattern = os.environ.get('search_pattern')
if not search_pattern:
    print("The 'search_pattern' environment variable is not defined")
    sys.exit(1)
Compiledpattern = re.compile(search_pattern)
# Iterate over each line passed to the script via standard input.
for InputData in sys.stdin:
    InputData = InputData.strip()
    # If the compiled pattern is discovered in the line, print it.
    if Compiledpattern.search(InputData):
        print(InputData)

~
~
~
"mapper_vaishp_grep.py" 17L, 562B 17,0-1 All
```

Fig 10.1 Mapper Grep

Reducer (reducer_vaishp_grep.py):

- It prints the line received from the mapper after filtering based on the search pattern.

A screenshot of a terminal window titled 'hadoop@Ubuntu: ~'. The window displays the code for a Python script named 'reducer_vaishp_grep.py'. The code is as follows:

```
import sys

# Iterating through each line from the standard SInput.
for InputData in sys.stdin:
    InputData = InputData.strip()
    print(InputData)
    print("-----")
print("-----")
```

The terminal shows the first few lines of the script, with the rest of the code being truncated by a vertical scrollbar on the right. The status bar at the bottom of the terminal indicates the file name 'reducer_vaishp_grep.py', its size '8L, 381B', and the current line '8,0-1'.

Fig 10.2 Reducer Grep

Test Case 1 “sim” as search pattern:**Command to run Map Reduce:**

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_grep.py,reducer_vaishp_grep.py -mapper 'python3 mapper_vaishp_grep.py' -reducer
'python3 reducer_vaishp_grep.py' -input /input/sample.log -output /output/result2grep -cmdenv
search_pattern="sim"
```

Logs:

- Below are the logs obtained after running the above command.

```

hadoop@Ubuntu: ~
hadoop@Ubuntu:~$ hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files mapper_vaishp_grep.py, reducer_vaishp_grep.py -mapre
r 'python3 mapper_vaishp_grep.py' -reducer 'python3 reducer_vaishp_grep.py' -input /input/sample.log -output /output/result2grep -cmdenv search_patte
rn="sim"
packageJobJar: [/tmp/hadoop-unjar3683995106511075064/] [] /tmp/streamjob4304245216799033749.jar tmpDir=null
2024-03-09 18:13:46,356 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 18:13:46,549 INFO client.DefaultNoHARMFaloverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2024-03-09 18:13:46,801 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/hadoop/.staging/job_171002090
2181_0007
2024-03-09 18:13:47,592 INFO mapred.FileInputFormat: Total input files to process : 1
2024-03-09 18:13:48,094 INFO mapreduce.JobSubmitter: number of splits:2
2024-03-09 18:13:48,236 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1710020902181_0007
2024-03-09 18:13:48,236 INFO mapreduce.JobSubmitter: Executing with tokens: []
2024-03-09 18:13:48,433 INFO conf.Configuration: resource-types.xml not found
2024-03-09 18:13:48,434 INFO resource.ResourceUtils: Unable to find 'resource.types.xml'.
2024-03-09 18:13:48,548 INFO impl.VarnClientImpl: Submitted application application_1710020902181_0007
2024-03-09 18:13:48,592 INFO mapreduce.Job: The url to track the job: http://Ubuntu.myguest.virtualbox.org:8088/proxy/application_1710020902181_0007/
2024-03-09 18:13:48,793 INFO mapreduce.Job: Running job: job_1710020902181_0007
2024-03-09 18:13:54,793 INFO mapreduce.Job: Job job_1710020902181_0007 running in uber mode : false
2024-03-09 18:13:54,794 INFO mapreduce.Job: map 0% reduce 0%
2024-03-09 18:13:59,903 INFO mapreduce.Job: map 100% reduce 0%
2024-03-09 18:14:04,958 INFO mapreduce.Job: map 100% reduce 100%
2024-03-09 18:14:06,989 INFO mapreduce.Job: Job job_1710020902181_0007 completed successfully
2024-03-09 18:14:07,075 INFO mapreduce.Job: Counters: 54
File System Counters
  FILE: Number of bytes read=2810
  FILE: Number of bytes written=845082
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=106676
  HDFS: Number of bytes written=4064
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0
Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=6070
  Total time spent by all reduces in occupied slots (ms)=2229
  Total time spent by all map tasks (ms)=6070
  Total time spent by all reduce tasks (ms)=2229
  Total vcore-milliseconds taken by all map tasks=6070
  Total vcore-milliseconds taken by all reduce tasks=2229
  Total megabyte-milliseconds taken by all map tasks=6215680
  Total megabyte-milliseconds taken by all reduce tasks=2282496

```

Fig 10.3 Logs for Grep

Command to check output: `hdfs dfs -cat /output/result2grep/part-00000`

Final Output Test Case 1 Grep:

```

hadoop@Ubuntu:~$ hdfs dfs -cat /output/result2grep/part-00000
178.253.33.51 - - [22/Jun/2019:03:56:19 +0330] "GET /n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT HTTP/1.1" 200 20406 "https://www.zanbil.ir/n/filter/p5767%2Ct156?name=%D9%85%D8%A7%D8%B4%D8%B8C%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD&productType=electric-shavers" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----
178.253.33.51 - - [22/Jun/2019:03:56:20 +0330] "GET /settings/logo HTTP/1.1" 200 4120 "https://www.zanbil.ir/n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----
178.253.33.51 - - [22/Jun/2019:03:56:21 +0330] "GET /image/32574?name=pr465at.jpg&wh=max HTTP/1.1" 200 28825 "https://www.zanbil.ir/n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----
178.253.33.51 - - [22/Jun/2019:03:56:21 +0330] "GET /image/32574?name=pr465at3.jpg&wh=max HTTP/1.1" 200 24001 "https://www.zanbil.ir/n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----
178.253.33.51 - - [22/Jun/2019:03:56:23 +0330] "GET /image/32574?name=pr465at1.jpg&wh=max HTTP/1.1" 200 47078 "https://www.zanbil.ir/n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----
178.253.33.51 - - [22/Jun/2019:03:56:24 +0330] "GET /image/32574?name=pr465at2.jpg&wh=max HTTP/1.1" 200 46807 "https://www.zanbil.ir/n/product/32574/62991/%D9%85%D8%A7%D8%B4%D8%B8%D9%86-%D8%A7%D8%B5%D9%84%D8%A7%D8%AD-%D8%B5%D9%88%D8%B1%D8%AA-%D9%8B%D8%B1%D9%80%D8%B3%D9%84%D8%B8C-%D9%85%D8%AF%D9%84-PR465AT" Mozilla/5.0 (Linux; Android 5.1; HTC Desire 728 dual sim) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/71.0.3578.83 Mobile Safari/537.36" "-"
-----

```

Fig 10.4 Final Output Grep Test Case 1

Test Case 1 “preparation” as search pattern:**Command to run Map Reduce:**

```
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar -files
mapper_vaishp_grep.py, reducer_vaishp_grep.py -mapper 'python3 mapper_vaishp_grep.py' -
reducer 'python3 reducer_vaishp_grep.py' -input /input/sample.log -output
/output/result2greptest2 -cmdenv search_pattern="preparation"
```

Command to check output: `hdfs dfs -cat /output/result2greptest2/part-00000`

Final Output Test Case 2 Grep:

```
2024-03-09 18:19:35,683 INFO streaming.StreamJob: Output directory: /output/result2greptest2
hadoop@Ubuntu:~$ hdfs dfs -cat /output/result2greptest2/part-00000
5.211.97.39 - - [22/Jun/2019:03:56:50 +0330] "GET /m/browse/food-preparation/%D8%A2%D9%85%D8%A7%D8%AF%D9%87-%D8%B3%D8%A7%D8%B2%D8%8C-%D8%BA%D8%B0%D8%A7 HTTP/1.1" 200 16726 "https://www.zanbil.ir/m/browse/cooking-tools/%D9%BE%D8%AE%D8%AA-%D9%88-%D9%BE%D8%B2" "Mozilla/5.0 (iPhone; CPU iPhone OS 10_3_2 like Mac OS X) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.0 Mobile/14F89 Safari/602.1" "-"
-----
5.211.97.39 - - [22/Jun/2019:03:56:50 +0330] "GET /settings/logo HTTP/1.1" 200 4120 "https://www.zanbil.ir/m/browse/food-preparation/%D8%A2%D9%85%D8%A7%D8%AF%D9%87-%D8%B3%D8%A7%D8%B2%D8%8C-%D8%BA%D8%B0%D8%A7" "Mozilla/5.0 (iPhone; CPU iPhone OS 10_3_2 like Mac OS X) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.0 Mobile/14F89 Safari/602.1" "-"
-----
5.211.97.39 - - [22/Jun/2019:03:56:50 +0330] "HEAD /amp_preconnect_polyfill_404_or_other_error_expected_Do_not_worry_about_it?1548117000000 HTTP/1.1" 404 0 "https://www.zanbil.ir/m/browse/food-preparation/%D8%A2%D9%85%D8%A7%D8%AF%D9%87-%D8%B3%D8%A7%D8%B2%D8%8C-%D8%BA%D8%B0%D8%A7" "Mozilla/5.0 (iPhone; CPU iPhone OS 10_3_2 like Mac OS X) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.0 Mobile/14F89 Safari/602.1" "-"
-----
5.211.97.39 - - [22/Jun/2019:03:56:56 +0330] "GET /m/browse/meat-grinder/%D8%A6%D8%B1%D8%AE-%D8%AF%D9%88%D8%B4%D8%AA HTTP/1.1" 200 19543 "https://www.zanbil.ir/m/browse/food-preparation/%D8%A2%D9%85%D8%A7%D8%AF%D9%87-%D8%B3%D8%A7%D8%B2%D8%8C-%D8%BA%D8%B0%D8%A7" "Mozilla/5.0 (iPhone; CPU iPhone OS 10_3_2 like Mac OS X) AppleWebKit/603.2.4 (KHTML, like Gecko) Version/10.0 Mobile/14F89 Safari/602.1" "-"
-----
```

Fig 10.4 Final Output Grep Test Case 2

★ Testing the Fair Scheduler:

- To test the fair scheduler, I included the following properties in \$HADOOP_HOME/yarn-site.xml. The fair scheduler module is necessary to change the scheduler to fair.
- The properties are below (Fig 11.1 yarn-site.xml for fair scheduler)

```
<!-- YARN configuration for fair scheduling-->
<property>
  <name>yarn.resourcemanager.scheduler.class</name>
  <value>org.apache.hadoop.yarn.server.resourcemanager.scheduler.fair.FairScheduler</value>
</property>
<property>
  <name>yarn.scheduler.fair.allocation.file</name>
  <value>/home/hadoop/hadoop/etc/hadoop/fair-scheduler.xml</value>
</property>
</configuration>
~
~
~
~
~
~
~
~/hadoop/etc/hadoop/yarn-site.xml" 30L, 1129B 30,16 All
```

Fig 11.1 yarn-site.xml for fair scheduler

- Fair-sceduler.xml file is created with the below queues in it:
 1. Database Search
 2. Word Count
 3. Sort
 4. Grep

```
hadoop@Ubuntu: ~
<?xml version="1.0"?>
<allocations>
  <defaultQueueSchedulingPolicy>fair</defaultQueueSchedulingPolicy>
  <queue name="root">
    <queue name="DatabaseSearch">
      <schedulingPolicy>fair</schedulingPolicy>
      <weight>0.75</weight>
    </queue>
    <queue name="WordCount">
      <schedulingPolicy>fair</schedulingPolicy>
      <weight>0.50</weight>
    </queue>
    <queue name="Sort">
      <schedulingPolicy>fair</schedulingPolicy>
      <weight>0.25</weight>
    </queue>
    <queue name="Grep">
      <schedulingPolicy>fair</schedulingPolicy>
      <weight>0.25</weight>
    </queue>
  </queue>
</allocations>
~
"fair-sceduler.xml" 22L, 590B 22,14 All
```

Fig: fair-sceduler.xml

- Below are the queues created in Hadoop

The screenshot displays the Hadoop cluster scheduler interface. The top section shows the Hadoop logo and the title "NEW,NEW_SAVING,SUBMITTED,ACCEPTED,RUNNING Applications". Below this, there are several tables and sections:

- Cluster Metrics:** A table showing various metrics like Apps Submitted, Apps Pending, Apps Running, Apps Completed, Containers Running, Used Resources, Total Resources, and Reserved Resources.
- Cluster Nodes Metrics:** A table showing metrics for Active Nodes, Decommissioning Nodes, Decommissioned Nodes, Lost Nodes, and Unhealthy Nodes.
- User Metrics for dr.who:** A table showing metrics for Apps Submitted, Apps Pending, Apps Running, Apps Completed, Containers Running, Containers Pending, Containers Reserved, Memory Used, Memory Pending, and Memory Reserved.
- Scheduler Metrics:** A table showing metrics for Scheduler Type, Scheduling Resource Type, Minimum Allocation, Maximum Allocation, and Maximum Cluster App.
- Application Queues:** A section showing a legend for queue types (Steady Fair Share, Instantaneous Fair Share, Used, Used over fair share, Max Capacity) and a list of queues (root, root default, root WordCount, root Sort, root DatabaseSearch, root Grep).
- Table of Applications:** A table with columns: ID, User, Name, Application Type, Queue, Fair Share, StartTime, LaunchTime, FinishTime, State, FinalStatus, Running Containers, and Allocated CPU VCores. The table shows one application: application_1710031687159_0004, user hadoop, streamjob3967822967439444843.jar, MAPREDUCE, root.WordCount, 2048, 1710031758681, N/A, N/A, undefined, UNDEFINED, 0, 0.

- In fair-scheduler.xml, I increased the weight of database search due to its high resource consumption and initial failure.
- I gave the name Word Count slightly less weight than Database Search but more than the other two.
- Sort and Grep received equal weights because they were less bulky and executed quickly.
- All the above changes can be seen in the fair-scheduler code above.

★ Testing the Capacity Scheduler:

- To implement the capacity scheduler, I modified capacity-scheduler.xml.
- I configured Database Search, wordcount, sort, and grep queues in my capacity-scheduler.xml, as shown in the screenshot below.


```

<property>
  <name>yarn.scheduler.capacity.capacity.root.DatabaseSearch.capacity</name>
  <value>35</value>
  <description>DatabaseSearch queue target capacity.</description>
</property>

<property>
  <name>yarn.scheduler.capacity.capacity.root.WordCount.capacity</name>
  <value>25</value>
  <description>WordCount queue target capacity.</description>
</property>

<property>
  <name>yarn.scheduler.capacity.capacity.root.Sort.capacity</name>
  <value>25</value>
  <description>Sort queue target capacity.</description>
</property>

<property>
  <name>yarn.scheduler.capacity.capacity.root.Grep.capacity</name>
  <value>15</value>
  <description>Grep queue target capacity.</description>
</property>
</configuration>

```

"~/hadoop/etc/hadoop/capacity-scheduler.xml" 268L, 9888B 268,2 Bo

Fig: capacity-scheduler.xml

- I noticed that queues had reached their maximum capacity and became stuck, so I implemented priority handling in fair scheduling.

```

hadoop@Ubuntu: ~
# !/bin/bash

# Database Search Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=DatabaseSearch -D mapred.job.priority=VERY_HIGH \
-files mapper_vaishp_part2.py, reducer_vaishp_part2.py \
-mapper 'python3 mapper_vaishp_part2.py' \
-reducer 'python3 reducer_vaishp_part2.py' \
-input /input/sample.log \
-output /output/result2ds \
-cmdenv timewindow="01-02" &

# Word Count Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=WordCount -D mapred.job.priority=HIGH \
-files mapper_vaishp_wc.py, reducer_vaishp_wc.py \
-mapper 'python3 mapper_vaishp_wc.py' \
-reducer 'python3 reducer_vaishp_wc.py' \
-input /input/sample.log \
-output /output/result2wc &

# Sort Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=Sort -D mapred.job.priority=NORMAL \
-files mapper_vaishp_sort.py, reducer_vaishp_sort.py \
-mapper 'python3 mapper_vaishp_sort.py' \
-reducer 'python3 reducer_vaishp_sort.py' \
-input /input/sample.log \
-output /output/result2sort &

# Grep Job
hadoop jar $HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-D mapreduce.job.queueName=Grep -D mapred.job.priority=LOW \
-files mapper_vaishp_grep.py, reducer_vaishp_grep.py \
-mapper 'python3 mapper_vaishp_grep.py' \
-reducer 'python3 reducer_vaishp_grep.py' \
-input /input/sample.log \
-output /output/result2grep \
-cmdenv search_pattern="sim" &

wait

echo "-----ALL TASK COMPLETED-----"

```

Fig: runAll.sh with Priority Scheduling

- I saw the queues run smoothly under fair scheduling.

★ **Conclusion:**

I learned a lot about scheduling and the map-reduce framework, as well as Hadoop itself.

★ **References:**

All the videos provide by TAs:

- installation of Hadoop on Jetstream2 Ubuntu instance with streaming example.mp4
- attended setup of Ubuntu VM on VirtualBox (Windows).mp4
- installation of Hadoop on Ubuntu VM in VirtualBox (Windows).mp4